# INCLUYA CARÁTULA

## LABORATORIO PRÁCTICO

# Lab Semana2: Conceptos de Seguridad en Redes

ENSA v7.0 — Entorno de Doble Máquina Virtual

| Duración | Modalidad | VMs | Prácticas |
|---|---|---|---|
| 3 horas | En Línea / Virtual | 2 VMs en modo Bridge | 3 laboratorios activos |

**Propósito de esta Guía**

Este laboratorio reemplaza la lectura pasiva de la semana 2 con experiencias PRÁCTICAS y DIVERTIDAS. En lugar de leer sobre ataques de red, ¡los vas a observar en tiempo real en tu propio entorno virtual! La clase en línea se divide en 3 rondas de laboratorio con retos, hallazgos y evidencias que demostrarán tu asistencia y comprensión.

## TOPOLOGÍA DEL LABORATORIO — Configuración de VMs

Toda la actividad ocurre en TU propia computadora física. Necesitas:

- VirtualBox o VMware instalado en tu PC/Mac
- 2 VMs con tarjetas de red en modo BRIDGE (conectadas a tu red local real)
- **VM1** (Atacante/Analista): **Kali Linux** — ya incluye todas las herramientas
- **VM2** (Víctima): **Ubuntu Server 22.04** o Windows 10 (o cualquiera que tengas)

## 📋 Configuración Rápida de Red Bridge

En VirtualBox → Configuración de la VM → Red → Adaptador 1:

```
Conectado a:  Red en modo puente (Bridged Adapter)
Nombre:       [tu interfaz de red física, ej: Wi-Fi o Ethernet]
Modo promiscuo: Permitir todo (para captura de tráfico)
```

Verifica conectividad entre VMs:

```
# En VM1 (Kali), obtén la IP de VM2:
ip addr show       # ver tu propia IP
arp -a             # ver IPs vecinas en la red
ping <IP-de-VM2>   # confirmar conectividad
```

> 💡 **Tip: ¿No tienes Kali Linux?**
> Descarga la imagen OVA oficial en kali.org/get-kali → Virtual Machines. Se importa directo a VirtualBox en 5 minutos. Ya trae Wireshark, nmap, Metasploit y todo lo necesario.

# 🔬 PRÁCTICA 1 — Reconocimiento de Red y Escaneo de Puertos

| 🎯 Tema cubierto | ⏱ Tiempo estimado | 🔧 Herramientas |
|---|---|---|
| Agentes de amenaza, herramientas de ataque (3.2, 3.3) | 50 minutos | Nmap, Wireshark, ARP |

## 🎮 Misión: ¡Eres el nuevo analista de seguridad!

Tu empresa acaba de detectar tráfico sospechoso. Tu misión es mapear la red antes de que lo haga el atacante, usando exactamente las mismas herramientas que ellos usan.

## Fase 1A — Descubrimiento de Hosts con ARP y ICMP

- Abre una terminal en VM1 (Kali Linux)
- Identifica tu subred:

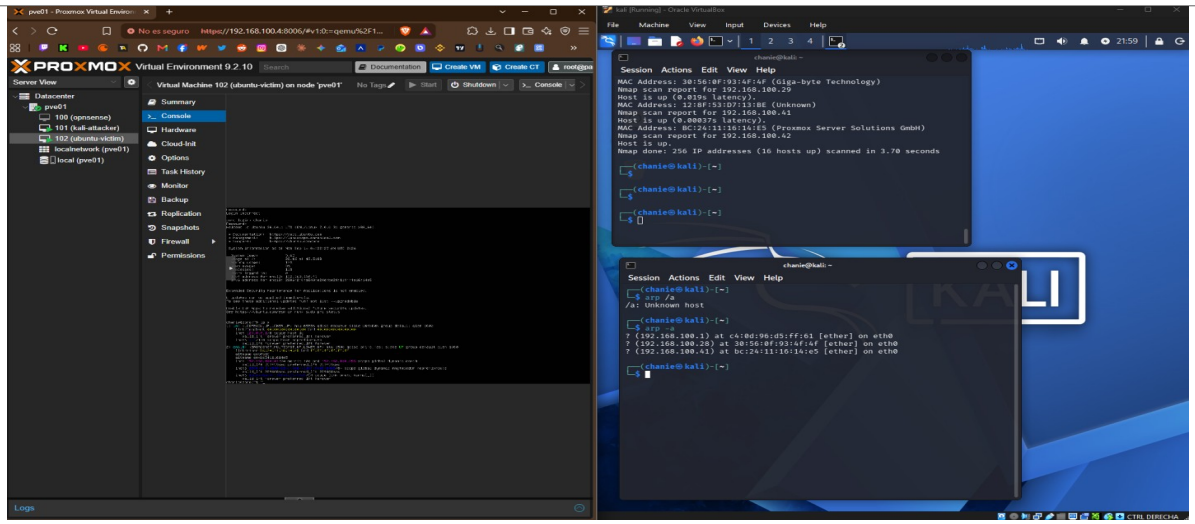
```
ip addr show
# Anota tu IP: 192.168.100.42 y tu máscara: /24
```

- Ejecuta un ping sweep para descubrir hosts vivos:

```
nmap -sn 192.168.1.0/24
# Cambia 192.168.1.0 por tu subred real  192.168.100.0/24
# Anota IPs encontradas:
192.168.100.1
192.168.100.4
192.168.100.9
192.168.100.18
192.168.100.42
```

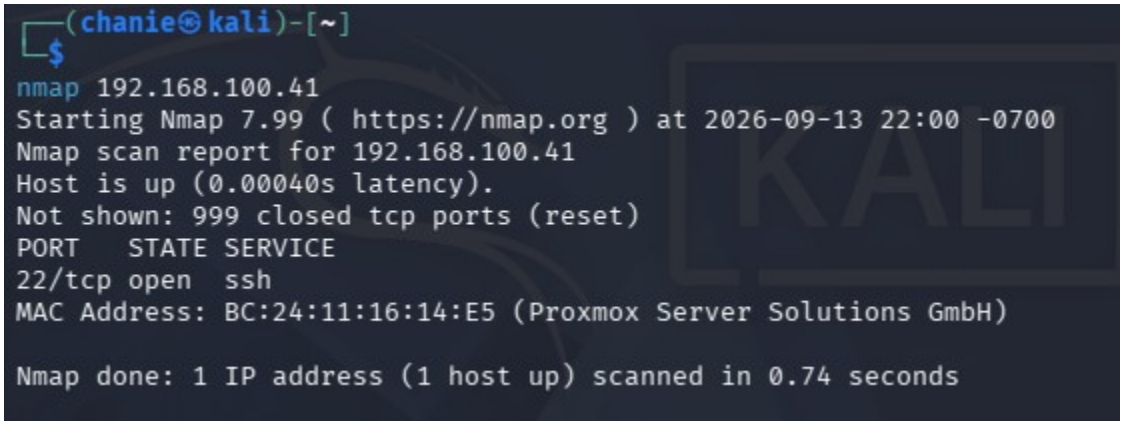
- Observa la tabla ARP después del escaneo:

```
arp -a
# ¿Qué dirección MAC tiene VM2? bc:24:11:16:14:e5
```



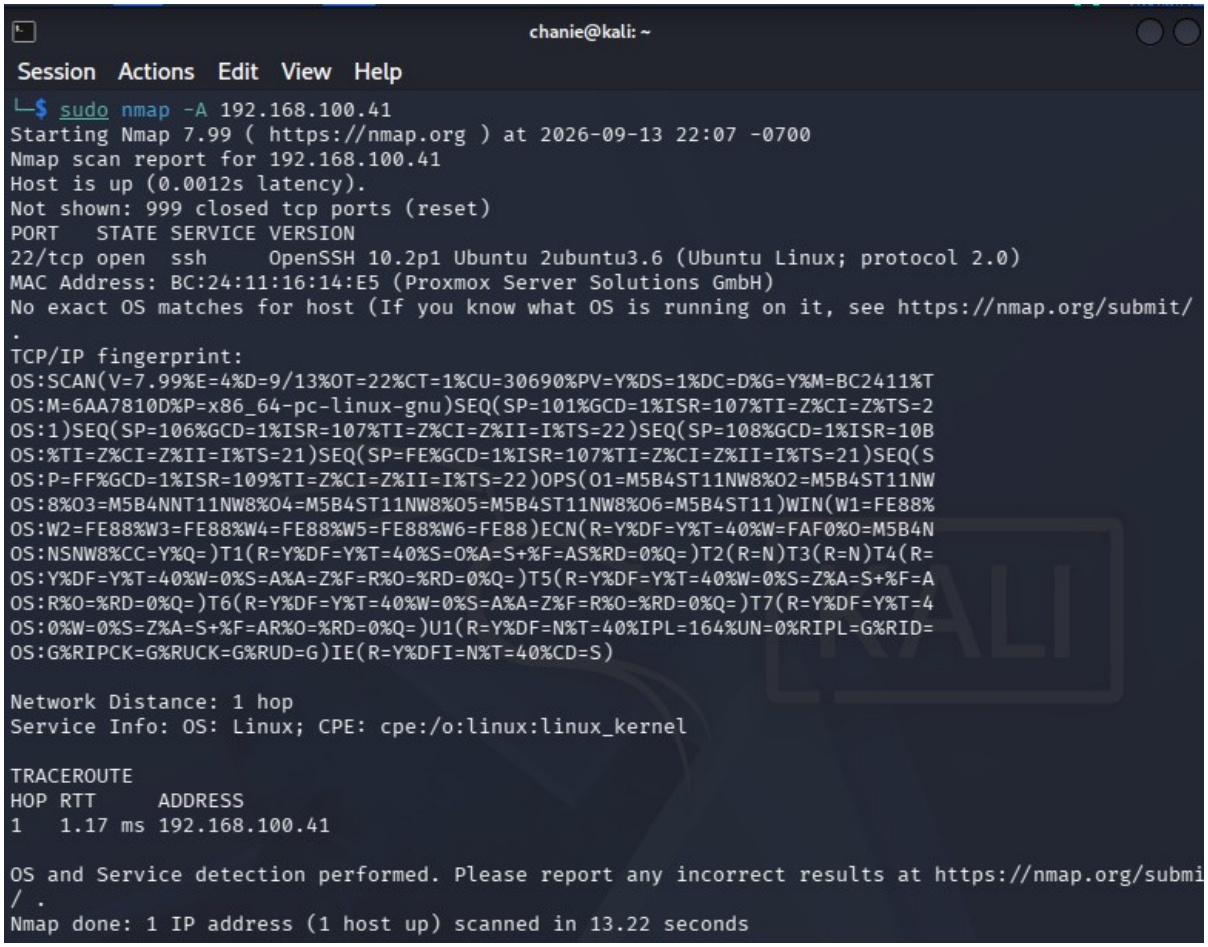

## Fase 1B — Escaneo de Puertos con Nmap

- Escaneo básico de puertos en VM2:



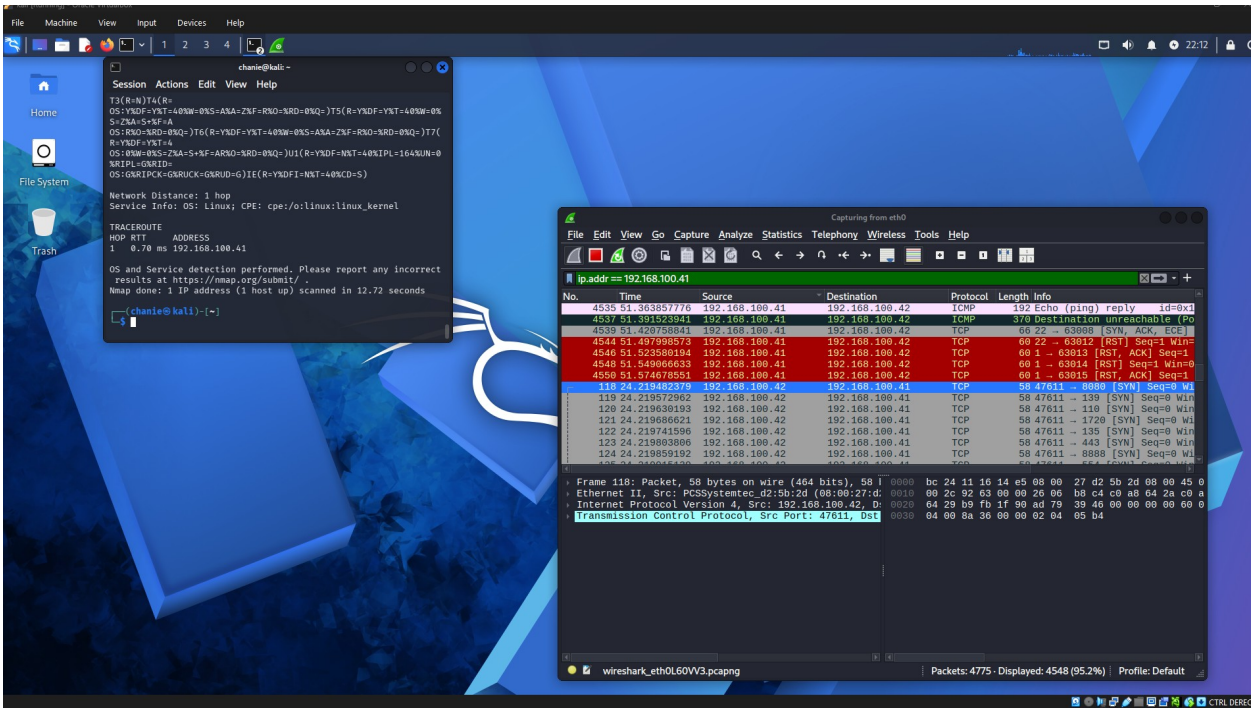

```
nmap -sV <IP-de-VM2>
# -sV detecta versiones de servicios
# Anota servicios encontrados:
#   Puerto   Servicio   Versión
#   22          tcp_      10.2p1 Ubuntu 2ubuntu3.6
```

- Escaneo agresivo (más información, más detectable):



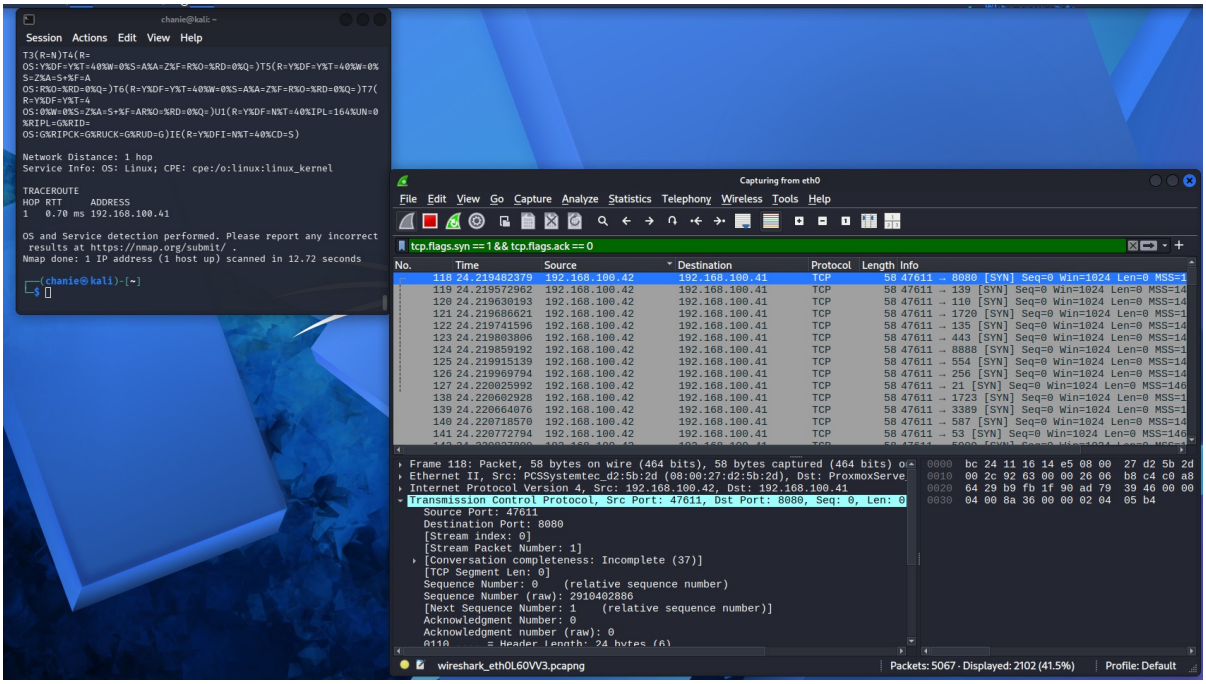

```
nmap -A <IP-de-VM2>
# ¿Qué sistema operativo detectó Nmap? Ubuntu Linux
```

- Captura el tráfico generado con Wireshark:




- Abre Wireshark en VM1
- Selecciona tu interfaz de red (eth0 o ens33)
- Aplica filtro: ip.addr == <IP-de-VM2>
- Ejecuta nuevamente el nmap y observa los paquetes en tiempo real
- Nmap -A -T4  (baremetal)

- En Wireshark, identifica paquetes SYN sin respuesta (puertos cerrados):



```
Filtro Wireshark: tcp.flags.syn == 1 && tcp.flags.ack == 0
# ¿Cuántos paquetes SYN ves? 2102
# Esto es la 'huella digital' de un escaneo — exactamente lo que los
# IDS/IPS están diseñados para detectar.
```

🏆 **Reto BONUS — El Escaneo Silencioso**
Los hackers avanzados usan técnicas para evadir la detección. Investiga y ejecuta: nmap -sS -T2 <IP-VM2> ¿Ves diferencia en el tráfico de Wireshark? ¿Por qué este escaneo es más difícil de detectar? Incluye tu análisis en la evidencia.

## 📸 EVIDENCIAS REQUERIDAS — Práctica 1

**¡IMPORTANTE — Sin evidencias = Sin asistencia!**
Las evidencias son tu 'firma de entrada' a la clase. Deben subirse al aula virtual antes de que termine la sesión. Capturas borrosas o de internet NO se aceptan — Wireshark y la terminal deben mostrar TUS IPs reales.

| # | Evidencia a entregar | Formato | Valor |
|---|---|---|---|
| 1 | Captura de pantalla del resultado de nmap -sn mostrando hosts descubiertos en tu subred (con TUS IPs) | Screenshot PNG | .5 pts |
| 2 | Captura de pantalla del resultado de nmap -sV en VM2, mostrando puertos y servicios detectados | Screenshot PNG | .5 pts |
| 3 | Captura de Wireshark mostrando paquetes SYN durante el escaneo (con filtro tcp.flags.syn==1 aplicado) | Screenshot PNG | .5 pts |
| 4 | Responde en documento: ¿Qué es un ataque de reconocimiento? ¿Por qué Nmap es legal usarlo en TU propia red pero no en redes ajenas? | PDF/Word | .5 pts |

# ⚔️ PRÁCTICA 2 — Captura y Análisis de Tráfico: ARP, DNS y TCP

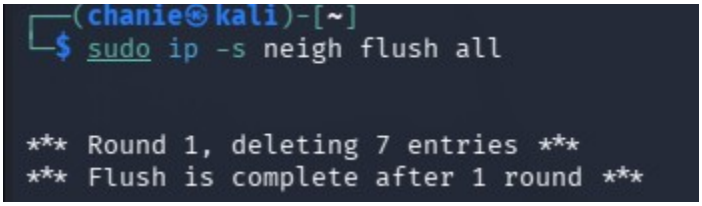
| 🎯 Temas cubiertos | ⏱ Tiempo estimado | 🔧 Herramientas |
|---|---|---|
| Vulnerabilidades TCP/UDP, Servicios IP: ARP y DNS (3.6, 3.7, 3.8) | 60 minutos | Wireshark, arpspoof / arping, dig, curl |

## 🦹 Misión: Intercepta el tráfico antes de que llegue al destino

En esta práctica vas a observar en TIEMPO REAL cómo fluyen protocolos vulnerables, exactamente como lo haría un atacante con herramientas de análisis de red. Aprenderás por qué ARP y DNS son tan peligrosos.

## Fase 2A — Observando el Protocolo ARP (la base del ARP Spoofing)

- En VM1, borra la caché ARP para observar el proceso desde cero:




```
sudo ip -s neigh flush all
arp -a   # Debe estar vacío o casi vacío
```

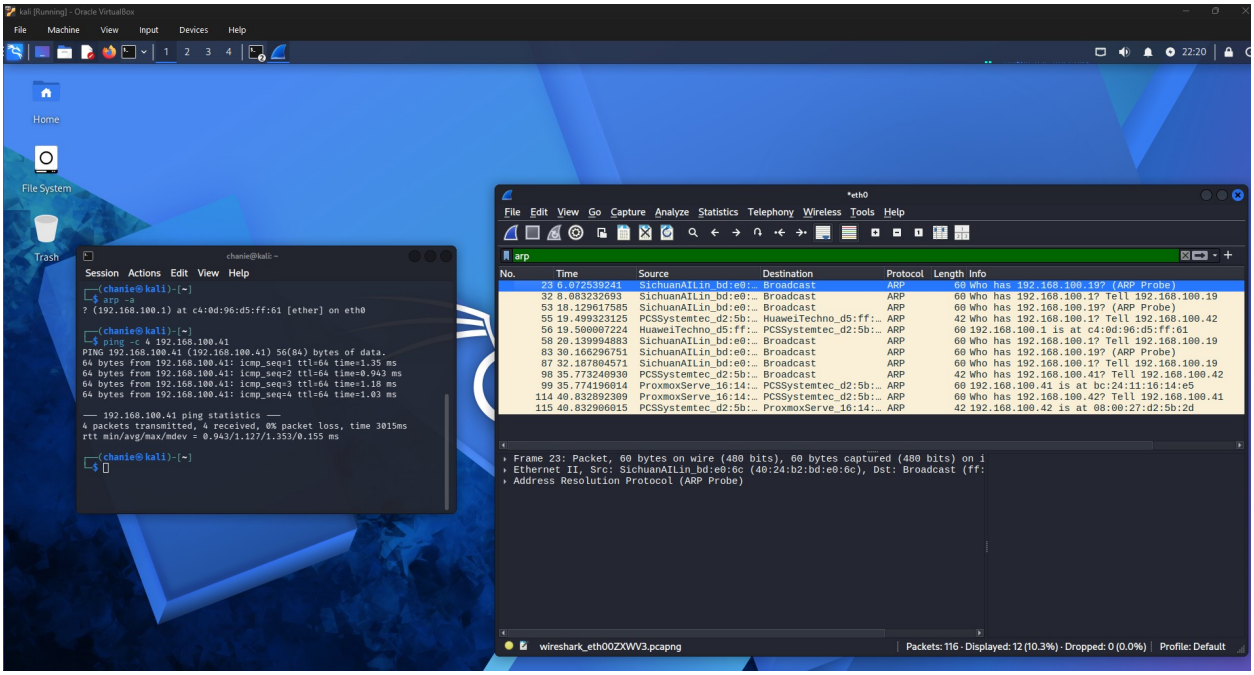
- Inicia Wireshark y filtra solo ARP:

```
Filtro: arp
# Deja Wireshark corriendo en background
```

- Desde VM1, hace ping a VM2:

```
ping -c 4 <IP-de-VM2>
```

- Pausa Wireshark y examina los paquetes ARP:
  - ¿Quién envió el ARP Request? (quién pregunta '¿quién tiene la IP X?')
  - 192.168.100.41
  - ¿Quién respondió con ARP Reply?
  - bc:24:11:16:14:e5
  - ¿Qué dirección MAC se asoció a la IP? Compara con: ip neigh show
  - bc:24:11:16:14:e5




- Ahora simula un 'ARP Gratuito' (Gratuitos ARP) — el mecanismo detrás del ARP Spoofing:

```
# Instala arping si no está disponible:
sudo apt install arping -y

# Envía un ARP gratuito anunciando tu MAC para la IP de VM2:
sudo arping -U -I eth0 -s <IP-de-VM2> <IP-de-VM2> -c 3
# Observa en Wireshark: ¡estás 'reclamando' una IP ajena!
# Esto es exactamente cómo funciona el ARP Spoofing/Poisoning
```

- Verifica el efecto en la tabla ARP de VM1:

```
arp -a
# ¿Cambió algo? Anota qué pasó:
    No se observó cambio en la tabla ARP local tras el ARP gratuito.
    Esto se debe a que el parámetro arp_accept del kernel Linux está
    deshabilitado por defecto (valor 0), lo que impide que el sistema
    actualice su caché ARP únicamente a partir de anuncios no
    solicitados. Esta es una mitigación de seguridad incorporada en
    el sistema operativo: un ataque real de ARP spoofing generalmente
    requiere enviar el anuncio falso directamente a la víctima
    objetivo (no auto-envenenar el propio caché del atacante), y su
    efectividad depende de si la víctima tiene esta protección
    deshabilitada o de usar técnicas más agresivas que sí logran
    sobrescribir entradas existentes.
```

```
┌──(chanie㉿kali)-[~]
└─$ arp -a
? (192.168.100.1) at c4:0d:96:d5:ff:61 [ether] on eth0
? (192.168.100.41) at bc:24:11:16:14:e5 [ether] on eth0

┌──(chanie㉿kali)-[~]
└─$ cat /proc/sys/net/ipv4/conf/eth0/arp_accept
0
```

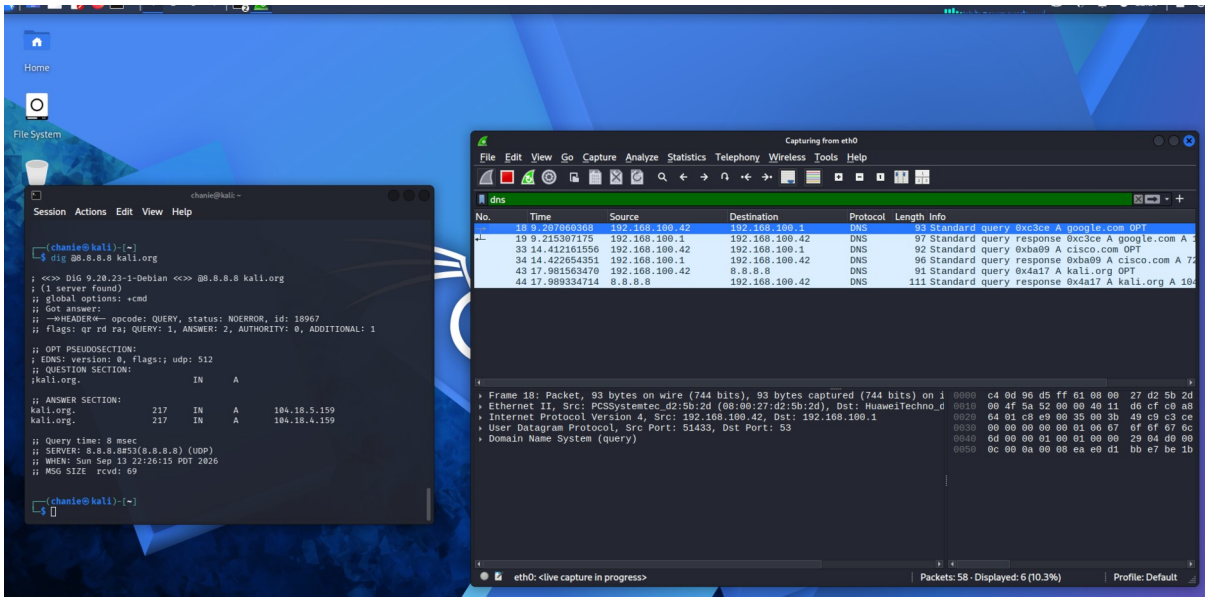
## Fase 2B — Análisis de Tráfico DNS

- Configura Wireshark con filtro DNS:

```
Filtro: dns
```

- Genera consultas DNS desde VM1:

```
dig google.com
dig cisco.com
dig @8.8.8.8 kali.org    # Consulta al DNS de Google directamente
```




- En Wireshark, examina cada paquete DNS y responde:
  - ¿Qué puerto usa DNS? UDP: _53 __ / TCP: _-__
  - ¿Cuál es el Transaction ID de tu consulta? _____**0xc3ce** _____
  - ¿Qué IP devolvió el servidor para google.com? : **`142.250.177.14`**

- ¿Cuántos registros A encontraste? _1

- Observa la estructura de una consulta DNS — esta información es la que se explota en DNS Tunneling:

```
# Genera una consulta DNS 'larga' (como haría un túnel DNS):
dig TXT $(python3 -c "print('A'*50)").google.com
# Observa en Wireshark la diferencia de tamaño
```

## Fase 2C — El Handshake TCP de 3 Vías (SYN, SYN-ACK, ACK)

- Asegúrate que VM2 tenga un servicio web o SSH activo (en Ubuntu: sudo systemctl start ssh)
- Filtra Wireshark solo para tu conexión:

```
Filtro: tcp && ip.addr == <IP-de-VM2>
```

- Inicia una conexión TCP desde VM1:

```
# Si VM2 tiene SSH (puerto 22):
ssh <usuario>@<IP-de-VM2>   # Conéctate y luego sal con: exit

# Si VM2 tiene web (puerto 80):
curl http://<IP-de-VM2>
```

- En Wireshark, localiza el handshake de 3 vías y documenta:

```
Paquete 1 SYN:     Número de Secuencia: Seq=0
Paquete 2 SYN-ACK: Número de Secuencia: Seq=0,
                   Número de ACK:       1
Paquete 3 ACK:     Número de ACK:       1Seq=1, Ack=1

# REFLEXIÓN: ¿En qué paso exactamente falla un ataque TCP SYN Flood? 3
# El servidor espera el paquete #___ que nunca llega, llenando su ___
```

**🏆 Reto BONUS — Detecta el SYN Flood con Wireshark**
Usa hping3 para simular un mini SYN Flood SOLO contra tu propia VM2: sudo hping3 -S --flood -p 80 <IP-de-VM2>. Observa en Wireshark cómo se ve el ataque. DETÉN el ataque en 5 segundos (Ctrl+C). ¿Cuántos paquetes SYN por segundo generaste? Incluye la captura como evidencia bonus.

## 📸 EVIDENCIAS REQUERIDAS — Práctica 2

| # | Evidencia a entregar | Formato | Valor |
|---|---|---|---|
| **1** | Wireshark mostrando paquetes ARP Request y ARP Reply durante el ping a VM2, con las MACs visibles | Screenshot PNG | **1 pts** |
| **2** | Terminal mostrando el resultado de arping -U y la tabla arp -a antes y después del comando | Screenshot PNG | **1 pts** |
| **3** | Wireshark con tráfico DNS: muestra una consulta y su respuesta, con el Transaction ID visible | Screenshot PNG | **.5 pts** |
| **4** | Wireshark mostrando el handshake TCP de 3 vías completo (SYN → SYN-ACK → ACK) con números de secuencia anotados | Screenshot PNG | **.4 pts** |
| **5** | Responde: ¿Por qué ARP es tan vulnerable? ¿Qué mecanismo hace que el envenenamiento de caché ARP sea posible? | PDF/Word 150 palabras | **.1 pts** |

# 🔐 PRÁCTICA 3 — Criptografía en Acción: Hash, Cifrado y TLS

| 🎯 Temas cubiertos | ⏱ Tiempo estimado | 🔧 Herramientas |
|---|---|---|
| Criptografía: Hash, Cifrado Simétrico/Asimétrico, CIA (3.9, 3.10) | 50 minutos | md5sum, sha256sum, openssl, Wireshark TLS |

## 🔑 Misión: Demuestra cómo funciona la criptografía moderna

Nada como ver la criptografía funcionar en vivo. En esta práctica vas a generar hashes, cifrar archivos con AES, crear un par de claves RSA, y ver en Wireshark la diferencia entre HTTP (texto claro) y HTTPS (cifrado).

## Fase 3A — Funciones Hash (MD5, SHA-256)

- Crea un archivo de texto y calcula su hash:

```
echo 'Hola Mundo - ENSA Modulo 3' > mensaje.txt
md5sum mensaje.txt
sha256sum mensaje.txt

# Anota los hash:
MD5:    383cb22a2ff4afff9375a7eff1bda3cd
SHA256: 342bf6190241ba3c1ae66988f28e0c50505a4ded1910f73874d153e4acdbecba
```

- Modifica UNA sola letra del archivo y recalcula — observa el efecto avalancha:

```
echo 'Hola mundo - ENSA Modulo 3' > mensaje2.txt
# (cambié 'M' mayúscula por 'm' minúscula)
md5sum mensaje2.txt
sha256sum mensaje2.txt

# ¿Son similares los hashes? no ¿Por qué? un hash es de una sola vía, sirve para verificar integridad, no para recuperar el contenido original.
```

- Verifica la integridad de un archivo descargado (simula verificación de software):

```
# Descarga un archivo y verifica su hash oficial
# Esto es exactamente lo que haces cuando verificas una ISO de Kali
echo 'Archivo original' > original.txt
sha256sum original.txt > original.txt.sha256
sha256sum -c original.txt.sha256
# Resultado: original.txt: OK
```

## Fase 3B — Cifrado Simétrico con AES (OpenSSL)

- Cifra un archivo con AES-256:

```
# Crea el archivo a cifrar
echo 'Información confidencial: contraseña=admin123' > secreto.txt

# Cifra con AES-256-CBC
openssl enc -aes-256-cbc -pbkdf2 -in secreto.txt -out secreto.enc -k
MiClave123

# Verifica que está cifrado (debe ser ilegible):
cat secreto.enc
xxd secreto.enc | head -5   # Ver en hexadecimal
```

- Descifra el archivo:

```
openssl enc -aes-256-cbc -pbkdf2 -d -in secreto.enc -out recuperado.txt -k
MiClave123
cat recuperado.txt
# ¿Se recuperó el texto original? __si__

# Intenta descifrar con clave incorrecta:
openssl enc -aes-256-cbc -pbkdf2 -d -in secreto.enc -out fail.txt -k
ClaveWrong
# ¿Qué pasa? OpenSSL devuelve un error de descifrado (bad decrypt) o, en versiones
que no validan el padding, genera un archivo con contenido corrupto/ilegible. Esto
demuestra que sin la clave exacta es computacionalmente inviable recuperar el contenido.
```

- Observa el tamaño del archivo cifrado vs original:

```
ls -lh secreto.txt secreto.enc
# Original: 48 _ bytes   Cifrado:80 bytes
# ¿El cifrado AES cambia mucho el tamaño? 48→80 bytes, +32 bytes
```

## Fase 3C — Cifrado Asimétrico RSA (Par de Claves)

- Genera un par de claves RSA (simula lo que hace SSH o TLS):

```
# Generar clave privada RSA de 2048 bits
openssl genrsa -out privada.pem 2048

# Extraer la clave pública
openssl rsa -in privada.pem -pubout -out publica.pem
```

```
# Observa la estructura:
cat privada.pem | head -5
cat publica.pem
```

- Cifra con la clave PÚBLICA y descifra con la PRIVADA:

```
echo 'Mensaje secreto RSA' > msg.txt

# Cifrar (cualquiera con la clave pública puede cifrar)
openssl pkeyutl -encrypt -pubin -inkey publica.pem -in msg.txt -out
msg.enc

# Descifrar (SOLO quien tiene la clave privada puede descifrar)
openssl pkeyutl -decrypt -inkey privada.pem -in msg.enc -out msg_dec.txt
cat msg_dec.txt
```

- Calcula HMAC — autenticación de mensajes (protección contra Man-in-the-Middle):

```
echo 'Transferir 100 USD a Alex' > transaccion.txt

# Generar HMAC con clave compartida:
openssl dgst -sha256 -hmac 'ClaveSecretaCompartida' transaccion.txt
# HMAC:SHA2-256(transaccion.txt) =
0ad9605a64cb1912c6773ec1148bae2b99c81c06fd8df29540b9fb9be4fe01a HMAC-SHA2-
256(transaccion_modificada.txt) =
6b0d8756b885194d414eaabc58d08e29dd845b6432b2973efcd2090c736a0dc5
# Si alguien modifica el mensaje:Sí, totalmente distinto
echo 'Transferir 10000 USD a Hacker' > transaccion_modificada.txt
openssl dgst -sha256 -hmac 'ClaveSecretaCompartida'
transaccion_modificada.txt
# HMAC diferente! El receptor detectaría la manipulación.
```

## Fase 3D — TLS en Acción con Wireshark

- Configura Wireshark con filtro para ver tráfico web:

```
Filtro: tls || http
```

- Genera tráfico HTTP (sin cifrar) y HTTPS (cifrado):

```
# HTTP - texto claro (ya casi no existe, pero úsalo con sitios de prueba):
curl http://example.com -v 2>&1 | head -20

# HTTPS - cifrado TLS:
curl https://example.com -v 2>&1 | head -20
```

- En Wireshark, compara los dos tipos de tráfico:

- HTTP: ¿Puedes leer el contenido de la respuesta en Wireshark? SÍ se ve el `GET / HTTP/1.1`
- HTTPS: ¿Puedes leer el contenido?/ NO
- TLS Handshake: ¿Qué versión de TLS se negoció? TLSv1.3 (OUT), TLS handshake, Client hello (1)
- ¿En qué paquete se hace el 'Client Hello'? *El Client Hello corresponde al primer mensaje del handshake TLS (tipo 1), enviado inmediatamente después de establecida la conexión TCP con el puerto 443 — sería visible en Wireshark como el primer paquete con el filtro `tls.handshake.type==1` tras el ACK final del three-way handshake TCP.*
-

🏆 **Reto BONUS — Crack un Hash MD5**
Hashcat y John the Ripper son herramientas legítimas de auditoría. En Kali: echo -n 'password' | md5sum → obtendrás 5f4dcc3b5aa765d61d8327deb882cf99. Usa hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt.gz para intentar crackearlo. ¿Cuánto tarda? ¿Qué dice esto sobre por qué MD5 ya NO se usa para contraseñas?

## 📸 EVIDENCIAS REQUERIDAS — Práctica 3

| # | Evidencia a entregar | Formato | Valor |
|---|---|---|---|
| **1** | Terminal mostrando los hashes MD5 y SHA256 del archivo original y del archivo modificado (lado a lado para ver el efecto avalancha) | Screenshot PNG | **1 pts** |
| **2** | Terminal mostrando el archivo secreto.txt (legible) vs secreto.enc (cifrado) — ambos con cat o xxd | Screenshot PNG | **1 pts** |
| **3** | Terminal mostrando la generación del par RSA (privada.pem y publica.pem) y el proceso de cifrado/descifrado exitoso | Screenshot PNG | **.5 pts** |
| **4** | Wireshark comparando: un paquete HTTP con contenido legible VS un paquete HTTPS/TLS con datos cifrados (2 capturas) | 2 Screenshots PNG | **.4 pts** |
| **5** | Reflexión escrita: Explica con tus palabras la diferencia entre cifrado simétrico (AES) y asimétrico (RSA). ¿Cuándo usarías cada uno? | PDF/Word 200 palabras | **.1 pts** |

# CONCLUSIONES

Establecer su postura con respecto al riesgo que corre la información al no tener regulación o políticas de seguridad para la prevención del delito cibernético y su punto de vista desde la perspectiva legal con respecto a que posibles leyes se estarían incumpliendo en caso de que este escenario fuera un ataque real.

# 📦 RESUMEN DE ENTREGA Y CRITERIOS DE ASISTENCIA

> 🎯 **¿Cómo se registra tu asistencia?**
> Tu asistencia a esta clase se verifica mediante las evidencias de laboratorio. No basta con conectarte a la sesión — debes demostrar que HICISTE las prácticas. Cada práctica equivale a un 'check' de asistencia. Si entregas las 3 prácticas completas, tienes asistencia completa.

## 📋 Tabla Resumen de Evidencias

| Práctica | Evidencias | Puntos totales |
|---|---|---|
| 🔬 Práctica 1 — Reconocimiento y Escaneo | 4 evidencias (3 capturas + 1 reflexión) | **2 pts** |
| ⚔️ Práctica 2 — ARP, DNS y TCP | 4 evidencias (4 capturas Wireshark) | **3 pts** |
| 🔐 Práctica 3 — Criptografía | 4 evidencias (2 capturas terminal + 2 Wireshark) | **3 pts** |
| Calidad de la evidencia | IP's, Mac Address, Protocolo, Puertos etc. | **1 pts** |
| Reflexión y análisis | Relación los riegos con argumentos legales críticos | **.5 pts** |
| Referencias | APAv7 completas | **.5 pts** |
| **Total Actividad2** | **3 retos** | **10 pts** |

## 📁 Cómo Entregar tus Evidencias

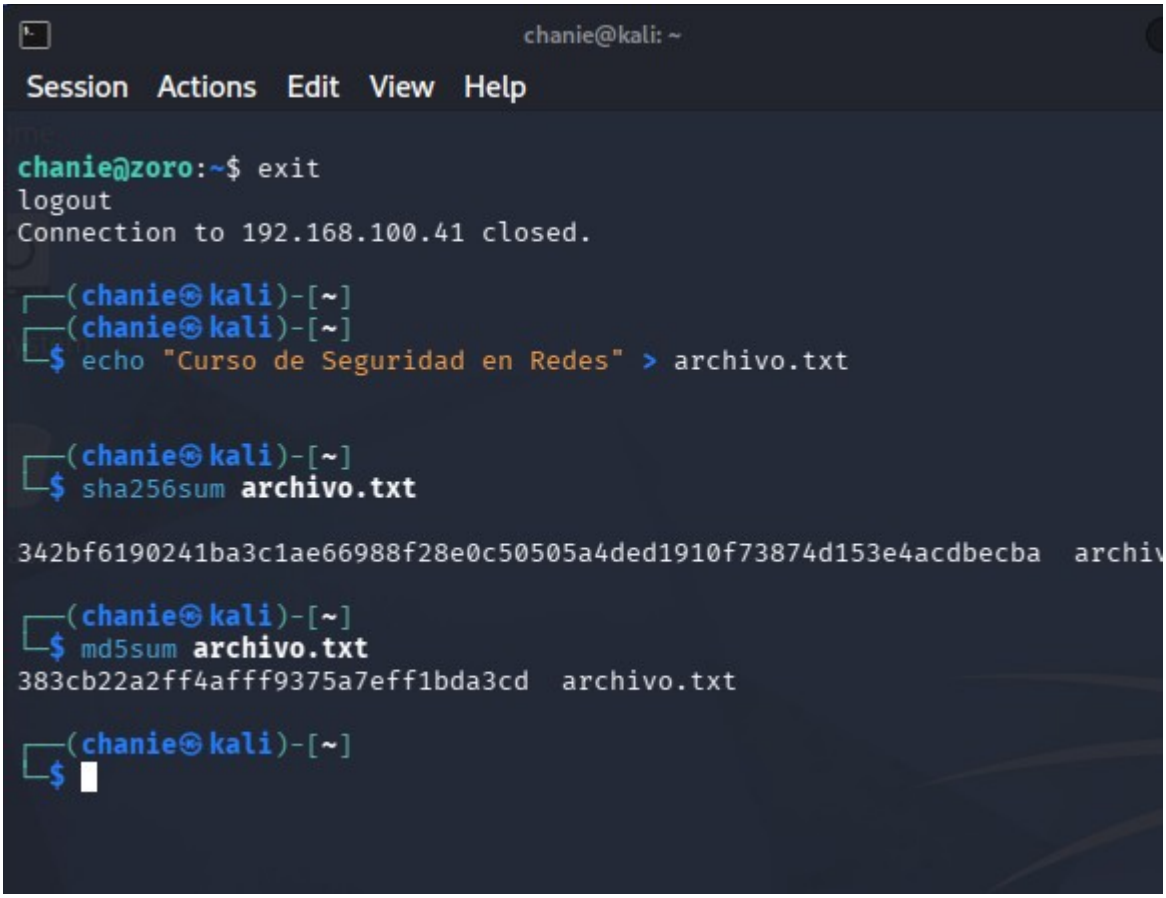
1. Crea una carpeta con tu nombre: NombreApellido_Lab_Semana2
2. Dentro, crea 3 subcarpetas: Practica1, Practica2, Practica3
3. Guarda las capturas de pantalla con nombres descriptivos:

- P1_E1_nmap_discovery.png
- P1_E2_nmap_portscan.png
- P1_E3_wireshark_syn.png
- P1_E4_reflexion.pdf

- (y así para P2 y P3)

4. Comprime todo en un ZIP: NombreApellido_Lab_Semana2.zip
5. Súbelo a la plataforma en la sección de tareas, si tienes duda escríbeme a dulcevc@umad.edu.mx

> 🚫 **Conducta Ética — Declaración Obligatoria**
> Las herramientas de este laboratorio son de uso EXCLUSIVO en el entorno virtual controlado de tu propia máquina física. Usar Nmap, hping3, arpspoof o cualquier herramienta de análisis/ataque en redes ajenas sin autorización expresa es ILEGAL en México (Artículo 211 bis del Código Penal Federal) y en la mayoría de países. Al participar en este laboratorio, el estudiante declara que usará estas herramientas únicamente en su entorno local de VMs.



# HOJA DE RESPUESTAS DEL ESTUDIANTE

Nombre: _____

Fecha de la sesión: _____ Hora inicio: _____ Hora fin: _____

IP de VM1 (Kali): _____ IP de VM2 (Víctima): _____

## Práctica 1 — Respuestas

¿Cuántos hosts encontró nmap -sn en tu subred? _____

Lista los puertos abiertos encontrados en VM2:

Puerto: _____ Servicio: _____
Puerto: _____ Servicio: _____
Puerto: _____ Servicio: _____

¿Cuántos paquetes SYN sin respuesta contaste en Wireshark? _____

Reflexión: ¿Qué es un ataque de reconocimiento y por qué nmap es legal en tu propia red?

_____

_____

_____

_____

## Práctica 2 — Respuestas

MAC de VM2 según arp -a: _____

¿Qué sucedió con la tabla ARP después del arping -U?

_____
___

Transaction ID de la primera consulta DNS: _____

IP devuelta para google.com: _____

Número de secuencia del SYN (paquete 1 del handshake TCP):
_____

Reflexión: ¿Por qué ARP es tan vulnerable al spoofing?

_____
___
_____
___

## Práctica 3 — Respuestas

Hash MD5 de mensaje.txt: _____
Hash MD5 de mensaje2.txt (con 'm' minúscula): _____

¿El cifrado AES cambia significativamente el tamaño del archivo? _____

¿Puedes leer el tráfico HTTP en Wireshark? _____ ¿Y el HTTPS/TLS? _____

Versión de TLS negociada en la conexión HTTPS: _____

Reflexión: Diferencia entre cifrado simétrico y asimétrico:

_____
_____
_____